3.5 Equivalence Testing

When working with reference types, there are many different notions of what it means for one expression to be equal to another. At the lowest level, if a and b are reference variables, then expression a == b tests whether a and b refer to the same object (or if both are set to the **null** value).

However, for many types there is a higher-level notion of two variables being considered "equivalent" even if they do not actually refer to the same instance of the class. For example, we typically want to consider two String instances to be equivalent to each other if they represent the identical sequence of characters.

To support a broader notion of equivalence, all object types support a method named **equals**. Users of reference types should rely on the syntax a.equals(b), unless they have a specific need to test the more narrow notion of identity. The equals method is formally defined in the Object class, which serves as a superclass for all reference types, but that implementation reverts to returning the value of expression a == b. Defining a more meaningful notion of equivalence requires knowledge about a class and its representation.

The author of each class has a responsibility to provide an implementation of the equals method, which overrides the one inherited from Object, if there is a more relevant definition for the equivalence of two instances. For example, Java's String class redefines equals to test character-for-character equivalence.

Great care must be taken when overriding the notion of equality, as the consistency of Java's libraries depends upon the equals method defining what is known as an ***equivalence relation*** in mathematics, satisfying the following properties:

Treatment of null: For any nonnull reference variable x, the call x.equals(**null**) should return **false** (that is, nothing equals **null** except **null**).

Reflexivity: For any nonnull reference variable x, the call x.equals(x) should return **true** (that is, an object should equal itself).

Symmetry: For any nonnull reference variables x and y, the calls x.equals(y) and y.equals(x) should return the same value.

Transitivity: For any nonnull reference variables x, y, and z, if both calls x.equals(y) and y.equals(z) return **true**, then call x.equals(z) must return **true** as well.

While these properties may seem intuitive, it can be challenging to properly implement equals for some data structures, especially in an object-oriented context, with inheritance and generics. For most of the data structures in this book, we omit the implementation of a valid equals method (leaving it as an exercise). However, in this section, we consider the treatment of equivalence testing for both arrays and linked lists, including a concrete example of a proper implementation of the equals method for our SinglyLinkedList class.

3.5.1 Equivalence Testing with Arrays

As we mentioned in Section 1.3, arrays are a reference type in Java, but not technically a class. However, the java.util.Arrays class, introduced in Section 3.1.3, provides additional static methods that are useful when processing arrays. The following provides a summary of the treatment of equivalence for arrays, assuming that variables a and b refer to array objects:

a == b: Tests if a and b refer to the same underlying array instance.

a.equals(b): Interestingly, this is identical to a == b. Arrays are not a true class type and do not override the Object.equals method.

Arrays.equals(a,b): This provides a more intuitive notion of equivalence, returning **true** if the arrays have the same length and all pairs of corresponding elements are "equal" to each other. More specifically, if the array elements are primitives, then it uses the standard == to compare values. If elements of the arrays are a reference type, then it makes pairwise comparisons a[k].equals(b[k]) in evaluating the equivalence.

For most applications, the Arrays.equals behavior captures the appropriate notion of equivalence. However, there is an additional complication when using multidimensional arrays. The fact that two-dimensional arrays in Java are really one-dimensional arrays nested inside a common one-dimensional array raises an interesting issue with respect to how we think about ***compound objects***, which are objects—like a two-dimensional array—that are made up of other objects. In particular, it brings up the question of where a compound object begins and ends.

Thus, if we have a two-dimensional array, a, and another two-dimensional array, b, that has the same entries as a, we probably want to think that a is equal to b. But the one-dimensional arrays that make up the rows of a and b (such as a[0] and b[0]) are stored in different memory locations, even though they have the same internal content. Therefore, a call to the method java.util.Arrays.equals(a,b) will return **false** in this case, because it tests a[k].equals(b[k]), which invokes the Object class's definition of equals.

To support the more natural notion of multidimensional arrays being equal if they have equal contents, the class provides an additional method:

Arrays.deepEquals(a,b): Identical to Arrays.equals(a,b) except when the elements of a and b are themselves arrays, in which case it calls Arrays.deepEquals(a[k],b[k]) for corresponding entries, rather than a[k].equals(b[k]).

3.5.2 Equivalence Testing with Linked Lists

In this section, we develop an implementation of the equals method in the context of the SinglyLinkedList class of Section 3.2.1. Using a definition very similar to the treatment of arrays by the java.util.Arrays.equals method, we consider two lists to be equivalent if they have the same length and contents that are element-by-element equivalent. We can evaluate such equivalence by simultaneously traversing two lists, verifying that x.equals(y) for each pair of corresponding elements x and y.

The implementation of the SinglyLinkedList.equals method is given in Code Fragment 3.19. Although we are focused on comparing two singly linked lists, the equals method must take an arbitrary Object as a parameter. We take a conservative approach, demanding that two objects be instances of the same class to have any possibility of equivalence. (For example, we do not consider a singly linked list to be equivalent to a doubly linked list with the same sequence of elements.) After ensuring, at line 2, that parameter o is nonnull, line 3 uses the getClass( ) method supported by all objects to test whether the two instances belong to the same class.

When reaching line 4, we have ensured that the parameter was an instance of the SinglyLinkedList class (or an appropriate subclass), and so we can safely cast it to a SinglyLinkedList, so that we may access its instance variables size and head. There is subtlety involving the treatment of Java's generics framework. Although our SinglyLinkedList class has a declared formal type parameter <E>, we cannot detect at runtime whether the other list has a matching type. (For those interested, look online for a discussion of ***erasure*** in Java.) So we revert to using a more classic approach with nonparameterized type SinglyLinkedList at line 4, and nonparameterized Node declarations at lines 6 and 7. If the two lists have incompatible types, this will be detected when calling the equals method on corresponding elements.

```
1   public boolean equals(Object o) {
2     if (o == null) return false;
3     if (getClass( ) != o.getClass( )) return false;
4     SinglyLinkedList other = (SinglyLinkedList) o;        // use nonparameterized type
5     if (size != other.size) return false;
6     Node walkA = head;                                    // traverse the primary list
7     Node walkB = other.head;                              // traverse the secondary list
8     while (walkA != null) {
9       if (!walkA.getElement( ).equals(walkB.getElement( ))) return false; //mismatch
10      walkA = walkA.getNext( );
11      walkB = walkB.getNext( );
12    }
13    return true;     // if we reach this, everything matched successfully
14  }
```

Code Fragment 3.19: Implementation of the SinglyLinkedList.equals method.